Project Development of Line Following Robot using Arduino ATMega328P

By: 3GB

Members	
B184	**Role/s**
Cañete, Phoebien D.	**HARDWARE DEV** & DOCUMENTATION
Hassan, Dja-ver Q.	**SOFTWARE DEV** & DOCUMENTATION
Listanco, Tristan E.	**SOFTWARE DEV** & DOCUMENTATION
Remo, Karl Vincent D.	**HARDWARE DEV** & DOCUMENTATION
Soronio, Mary Grace T.	**HARDWARE DEV** & DOCUMENTATION
Tanque, Shanice Reih H.	**SOFTWARE DEV** & DOCUMENTATION

Date Submitted: May 30, 2025
Group Rating: _____

I. Introduction

A. Overview of the project

This project focuses on the design and development of a line-following robot that utilizes the ATMega328P microcontroller. The robot's control algorithm is implemented in 8-bit AVR Assembly language, low-level hardware interfacing and control. An Infrared Proximity Sensor is utilized as the primary sensor for line detection, enabling the robot to follow black lines on a surface.

B. Objective of the line follower robot

1. To design and develop a line-following robot using the ATMega328P microcontroller integrated with a Infrared Proximity Sensor..
2. To evaluate the accuracy and response time of the robot when navigating through line tracks of varying complexity.
3. To design and implement a line-following algorithm written in 8-bit AVR Assembly language.

C. Brief Explanation of how the robot works

The line-following robot operates by utilizing the Infrared Proximity Sensor, which continuously scans the surface beneath it. When the module detects a black line ,the sensor sends signals to the microcontroller. The ATMega328P processes these signals using a control algorithm to adjust the motor speeds accordingly, allowing the robot to follow the path in an accurate manner.

II. Related Literature

A. Brief description of the similar technology

I. Ultrasonic Based Line Follower Robot

A Line Following Robot, which is a part of AGV (Automated Guided Vehicle), the autonomous robot in a simple term, can detect black or white lines to follow them. But research has been done to make them capable of following any color (Kaiser et al., 2014) . Light sensors based on LED or LDR and RFID based verification systems have been used to upgrade the line follower robot to that level.

Line follower is a smart autonomous robot that detects or follows a visible line embedded in the ground. The trail is predetermined and can be selected with a high contrast color or with a black line visible on the trail surface. Infrared sensors are used to detect these lines. A study by (Barua et al., 2020), develops a monitoring system that follows certain paths and can detect objects and edges using ultrasonic frequencies. If some object is put, a regular line follower will try to move and smash the obstacle. This prototype of a line follower robot tries to push the limit a little to overcome this issue. It has been built in a way that any obstacle in front of it can be identified. It will stop and will not pass until the barrier remains.

II. ATMega as Microcontroller for the Line Follower Robot

A study by He et al. (2014) demonstrated the development of embedded robot controllers based on ARM and FPGA. The adoption of ARM and FPGA can increase the flexibility of the controller greatly. ARM is mainly used to do algorithm calculation and communicate with PCs. FPGA is mainly used to generate the servo control signals and read the feedback signals from the servo system.

Additionally Latif et al.(2020) developed a line follower robot using ATmega32A microcontroller, the results show that the line follower robot can walk following the black line on the white floor and can display the situation on the LCD, at speeds of 90-150 RPM, the line follower robot can follow the path, while more than 150 rpm, the robot is not able to follow the path

A paper by Gadekar et al. (2021) explains the application of ATMega328P in smart systems. The first is 3-Phase Induction Motor Protection and Parameter Monitoring. The tremendously developed and increased industrial sector demands reliability along with the continuous and uninterrupted operations of the industrial appliances. The robust and easy to control feature of the Induction motor makes it the most suitable motor in the era of industrial development. As there are large demands for production to be increased. Another one is automated system for a poly-house that includes a soil moisture sensor that detects the soil's moisture content and, based on that, the system operates the pumps and the irrigation process is carried out, another parameter is the temperature within the poly-house, which will be sensed using a temperature sensor, and the temperature cooling mechanism will be activated.

III. IoT Based Approach Line Follower Robot

Another unique approach by Kokare et al. (2018) , An obstacle appears in the robot's path, the robot halts, activates a buzzer for alertness, and transmits a message wirelessly to a PC via ZigBee, stating “Obstacle is present.” A Sharp IR range sensor is utilized for obstacle detection, while the robot's motion is controlled using DC motors. The speed of these motors is regulated through Pulse Width Modulation (PWM). The study measured variations in voltage from the Sharp IR range sensor and the IR proximity sensor as the distance between the obstacle and the sensor changed. Additionally, it recorded voltage levels from a white line sensor when positioned over a white line and a black surface. The Received Signal Strength Indicator (RSSI) was also evaluated at various distances using XCTU software, revealing that RSSI values decrease as the distance increases.

IV. Infrared Proximity Sensor Applications

An infrared proximity sensor or IR Sensor is an electronic device that emits infrared light to detect objects and motion in Arduino projects. This active

IR sensor module is perfect for obstacle avoidance robots, motion detection systems, and proximity sensing applications. Whether you're building your first Arduino project or need a reliable sensor for advanced robotics, this complete tutorial covers wiring, coding, and troubleshooting with step-by-step instructions.(Das, 2022)

B. Importance of line follower in automation

I. Applications of Line Follower Robot in Medical Field

Several researches have been conducted in the medical field to aid nurses and doctors in the logistic aspect in their operation. A line following robot carrying medicine has been designed for providing the medicine to the patient whenever they need it (Jain et al., 2014). A two wheels balancing robot was developed by Ghani et al., 2011, which has the line following capability and for balancing it, they used infra-red distance sensor to solve the problem in inclination. Pakdaman & Sanaatiyan, 2009 has designed a small line following robot which uses IR sensors to detect the line drawn on the floor. That system was designed for the robot competition. Colak & Yildirim, 2009 has designed a line following robots to use in the shopping malls for entertainment. That system used a 4.8 cm wide black line to carry a maximum load of 400 kg. They also used a manual control with the help of a remote controller. A physical robot with 50 individual controls is generated by (Gomi & Ide, 1998 from which the ability and gait to lift the body can be improved. That robot can move its legs in forward motion and is tested in different conditions.

II. Applications of Line Follower Robot in Agricultural Field

The agriculture sector is the backbone of an economy which provides the basic ingredients to mankind and raw materials for industrialization. With the increasing number of the population over the world, the demand for agricultural products has also increased. In order to increase the production rate, irrigation techniques should be more efficient. A study by Rafi et al. (2016) finds that a line follower robot designed for irrigation-based applications can serve as a cost-effective and efficient solution for minimizing water loss. The proposed system operates autonomously, without the need for a human operator, and is fully portable. It incorporates a microcontroller, an on-board water reservoir, and an attached water pump. The robot is designed to irrigate any field with plants arranged along a predefined path. It possesses the capability to distinguish between movable objects and stationary plants, thereby reducing unnecessary water usage and ensuring targeted, autonomous watering.

III. Materials and Components

a. List of Hardware Components

1. (1) LiPo RC Battery 11.1V 60C 2250 mAh
2. (1) Arduino Uno R3 ATMega328P
3. (2) Infrared Proximity Sensor
4. (1) L298N H-Bridge Motor Driver
5. (1) Power Switch (SPST)
6. Jumper Wires
7. (1)12V Smart Car Robot Gear Motor
8. Stud and Spacers

b. List of Software and Programming Tools Used

1. PlatformIO IDE via VSCode Extension
2. Microchip Studio
3. EasyEDA Online PCB Maker

ii. Circuit Design and Assembly

a. Schematic Diagram: A symbolic representation of the circuit using standard electrical symbols. This should illustrate the logical connections between components

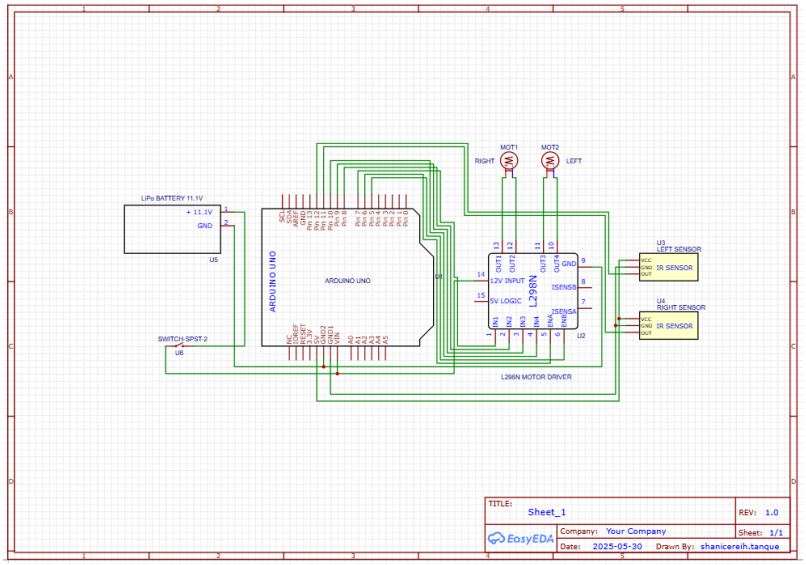


Figure 1: Schematic Diagram using EasyEDA Online PCB Software

Figure 1 provides a realistic depiction of the physical connections between the various hardware components of the Line Following Robot. This diagram illustrates the

actual wiring layout, showing how each module is interconnected to form the complete system.

Power Distribution:

The 11.1V LiPo RC Battery (U5) serves as the primary power source. It is connected through an SPST Power Switch (U6).

- From the switch, the positive (red wire) terminal connects to the VIN pin of the Arduino Uno and the 12V input terminal of the L298N H-Bridge Motor Driver.
- The negative (black wire) terminal of the battery connects to a common ground (GND) shared by the Arduino Uno and the L298N motor driver.

Control Unit - Arduino Uno (U1):

Power: It receives power via its VIN and GND pins from the switched battery output.

- **Sensor Inputs:**
  - The output (OUT) pin of the Left IR Proximity Sensor (U3) is connected to analog pin A0 of the Arduino.
  - The output (OUT) pin of the Right IR Proximity Sensor (U4) is connected to analog pin A1 of the Arduino.
-
- **Motor Control Outputs:** Digital pins D2, D3, D4, D5, D6, and D7 are used to send control signals to the L298N Motor Driver.
  - D2 is connected to ENB (Enable B) on the L298N.
  - D3 is connected to IN4 (Input 4) on the L298N.
  - D4 is connected to IN3 (Input 3) on the L298N.
  - D5 is connected to ENA (Enable A) on the L298N.
  - D6 is connected to IN2 (Input 2) on the L298N.
  - D7 is connected to IN1 (Input 1) on the L298N.
-

Motor Driver - L298N H-Bridge (U2):

The L298N module controls the speed and direction of the two DC motors.

- **Power Input:** Receives 12V and GND from the switched battery output.
- **Logic Power:** The 5V logic supply pin on the L298N is connected to the Arduino's 5V pin, ensuring both operate on the same logic level

and allowing the Arduino to power the L298N's logic circuits (or vice-versa if the L298N's onboard regulator is used).

- **Control Inputs:** Receives signals from Arduino digital pins D2-D7 as detailed above.
- **Motor Outputs:**
    - OUT1 and OUT2 terminals are connected to the Right DC Motor (MOT1).
    - OUT3 and OUT4 terminals are connected to the Left DC Motor (MOT2).

Sensors - Infrared Proximity Sensors (U3, U4):

Two IR Proximity Sensors are used for line detection.

- **Power:** Each sensor's VCC pin is connected to the Arduino's 5V pin, and its GND pin is connected to the Arduino's GND pin.
- **Signal Output:** The OUT pin of each sensor connects to an analog input pin on the Arduino (A0 for Left, A1 for Right) to provide line detection data.

b. Circuit Diagram: A realistic depiction of how the components are physically connected, including wiring details. This can be breadboard layout or PCB design

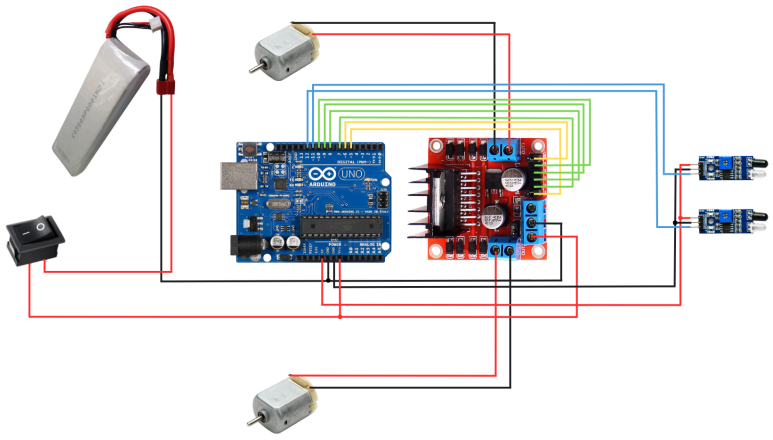


Figure 2: Circuit Diagram of the Line Following Robot

Iii. Programming & Algorithm

a.

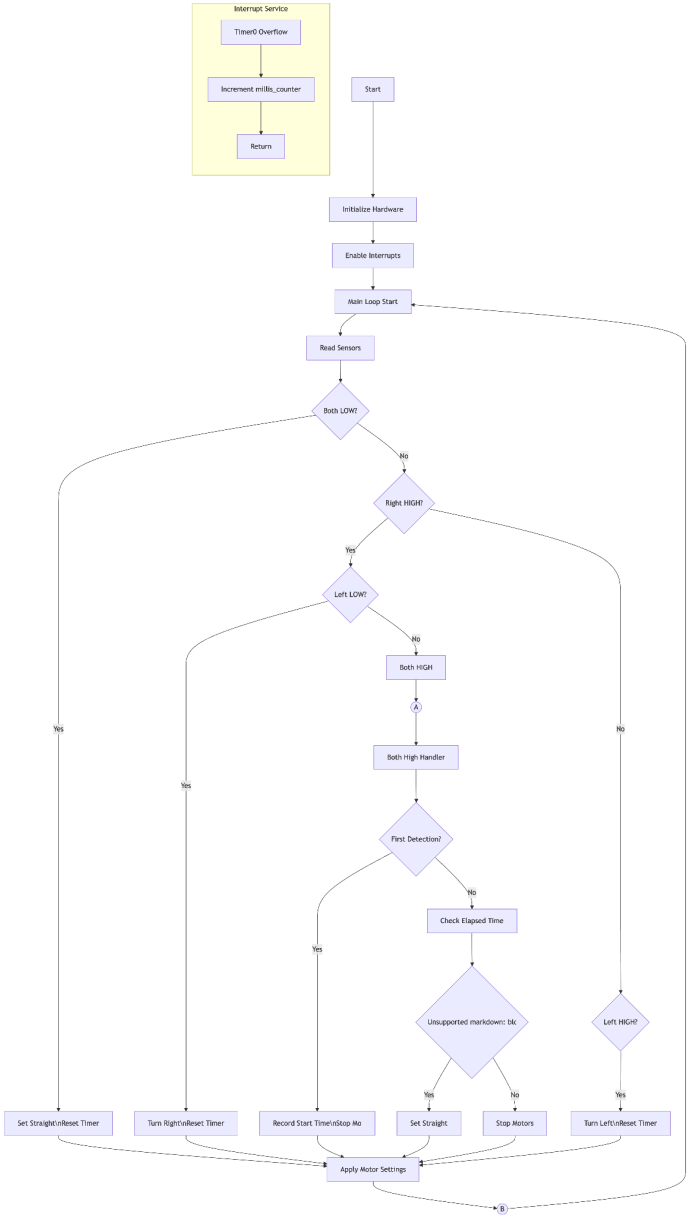


Figure 3: Flowchart Algorithm of the Line Following Robot

```
// System Initialization
INITIALIZE:
  Set stack pointer
  Clear millis_counter (32-bit)
  Clear both_black_start_time (32-bit)
  Configure Timer0:
    - 1ms interrupts (prescaler 64)
    - PWM output (prescaler 8)
  Set motor pins as outputs
  Set IR sensors as inputs
  Enable global interrupts
  Stop motors
  MAIN_LOOP()

// Main Control Loop
MAIN_LOOP:
  WHILE (true):
    right_ir = READ(PB3)   // 0=white, 1=black
    left_ir  = READ(PB4)   // 0=white, 1=black

    CASE (right_ir, left_ir):
      (0,0):  // Both white
        right_speed = +MOTOR_SPEED
        left_speed  = +MOTOR_SPEED
        both_black_start_time = 0

      (1,0):  // Right black, left white
        right_speed = -MOTOR_SPEED
        left_speed  = +MOTOR_SPEED
        both_black_start_time = 0

      (0,1):  // Left black, right white
        right_speed = +MOTOR_SPEED
        left_speed  = -MOTOR_SPEED
        both_black_start_time = 0

      (1,1):  // Both black (intersection)
        IF (both_black_start_time == 0):
          both_black_start_time = millis_counter
          right_speed = 0
          left_speed  = 0
        ELSE:
          elapsed = millis_counter -
both_black_start_time
          IF (elapsed >= 2000):
            right_speed = +MOTOR_SPEED
            left_speed  = +MOTOR_SPEED
          ELSE:
            right_speed = 0
            left_speed  = 0

    APPLY_MOTORS(right_speed, left_speed)

// Motor Control Function
APPLY_MOTORS(right_speed, left_speed):
  // Right motor
  IF (right_speed > 0):
    Set pins for forward
  ELSE IF (right_speed < 0):
    Set pins for backward
  ELSE:
    Set pins for stop
  Set PWM = |right_speed|

  // Left motor
  IF (left_speed > 0):
    Set pins for forward
  ELSE IF (left_speed < 0):
    Set pins for backward
  ELSE:
    Set pins for stop
  Set PWM = |left_speed|

// Timer Interrupt (1ms)
TIMER0_ISR:
  RELOAD timer (TCNT0 = 6)
  millis_counter++  // 32-bit increment
  RETURN
```

Figure 4: Pseudocode Equivalent of the Algorithm

Figure 3 and 4 shows the flowchart and pseudocode representation of the algorithm. Upon power is applied to the robot, the robot initializes its critical systems: It configures the stack pointer for stable program execution, clears its 32-bit timing counters (millis_counter and both_black_start_time), and sets up Timer0 for dual purposes. The motor control pins (PD5-PD7 and PB0-PB2) are configured as outputs, while the IR sensors (PB3 for right, PB4 for left) are set as inputs. After enabling global interrupts, the robot enters its perpetual main loop.

A LOW reading (0) indicates white surface, while HIGH (1) detects the black line. When both sensors read LOW, the robot moves straight forward at full speed (PWM 140), simultaneously resetting its intersection timer. If the right sensor detects the line (HIGH) while the left sees white (LOW), it executes a sharp right turn by reversing the right motor (-140) while driving the left motor forward.When the left sensor detects the line (HIGH) with the right on white (LOW), it performs a mirror-image left turn. Both turn scenarios also reset the intersection timer.

The critical intersection handling activates when both sensors detect black (HIGH). On initial detection, the robot records the current system time in both_black_start_time and halts completely then it calculates elapsed time using the background-maintained millis_counter. If under 2000ms, it maintains the stop; if 2000ms or more have passed, it proceeds straight through the intersection at full speed. Thus creates a 2-second pause at every crossing.

Positive speeds set Pin1 HIGH and Pin2 LOW for forward motion, negative speeds reverse these pins for backward rotation, while zero stops the motor. The absolute speed value determines PWM duty cycle on the enable pins (PD6 for right, PD5 for left), enabling precise speed control during turns. Meanwhile, the Timer0 interrupt service routine runs invisibly in the background, incrementing the 32-bit millis_counter every millisecond and maintaining precise timing for intersection pauses

iv. Testing and Debugging

a. Step-by-step testing process

The first step in testing is to ensure that all pins and wires are correctly connected and that all components are functioning properly and free from defects. This is essential to prevent potential damage or accidents during operation.

To test the robot's sensor detection and accuracy, we created a custom track using paper and tape. This setup allows us to observe how well the sensors detect the line and how the robot responds when turning. These tests help us fine-tune the turning speed, which is closely related to the motor speed and how the robot interprets sensor data. Based on the results, we adjust the values in the code to improve performance under different conditions.

Adjusting the motor speed in the code is critical. If the robot moves too quickly, the sensors may fail to accurately detect the line, leading to inaccurate or erratic data. This can cause the robot to veer off track and compromise its ability to follow the line properly.

Programming / Development Narrative:

The fundamental logic was first conceptualized and written in Embedded C. This allowed for quicker iteration and debugging of the control flow. The validated C code logic

was manually translated into 8-bit AVR Assembly language (line.S). The resulting assembly code was compiled and flashed onto the ATMega328P microcontroller using the PlatformIO IDE, integrated as an extension within Visual Studio Code. PlatformIO served as an effective build system, managing the assembly process and the interaction with the microcontroller.

The PlatformIO project typically involves a main C++ file (e.g., main.cpp) which acts as a minimal wrapper or entry point, with the core line-following algorithm residing within the separate assembly file (line.S) PlatformIO offers more robust support for mixed-language projects and helps minimize potential conflicts or overhead from unused Arduino core libraries

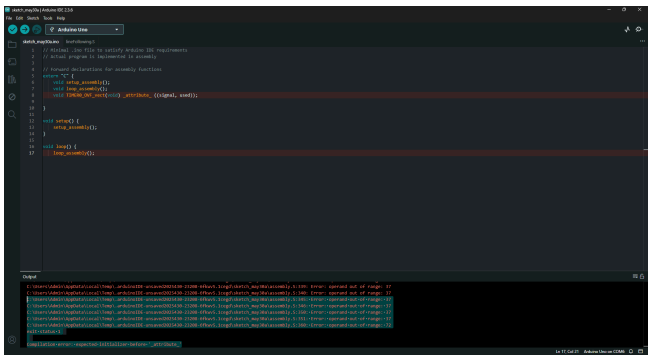
b. Common issues and solutions
   Motor jitter/no movement:
   Cause: Insufficient current from L298N's 5V regulator.
   Fix: Power Arduino via USB during tests or install external 5V buck converter.
   Action : Using the LiPo 11.1V Battery to sustain performance under heavy load / sufficiently supply power to the whole components of the robot

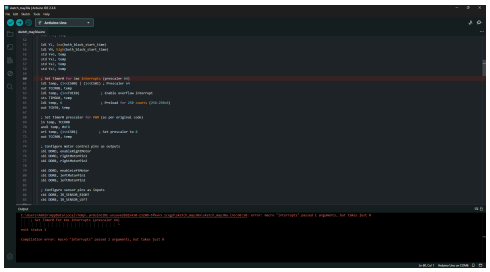
   Kanang issue about sa sensor, dili sha ka properly detect cuz either need pajud sha ipaduol sa floor surface, uneven surface and dili sakto iyang calibration sa physical hardware(physical hardware?? HASHAHA)

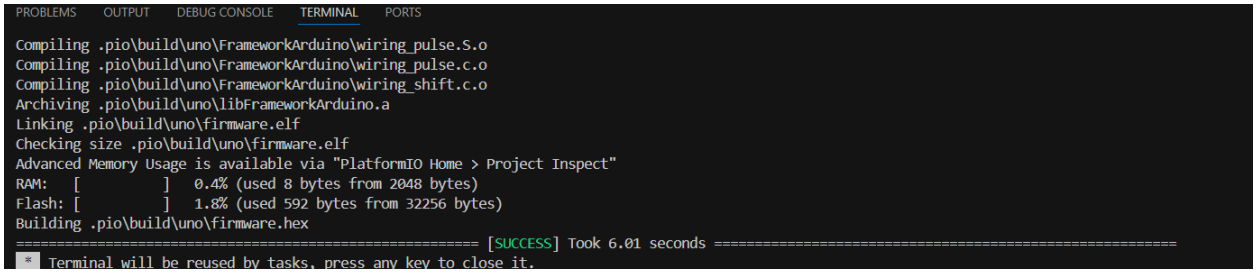
c. Performance Evaluation
   -

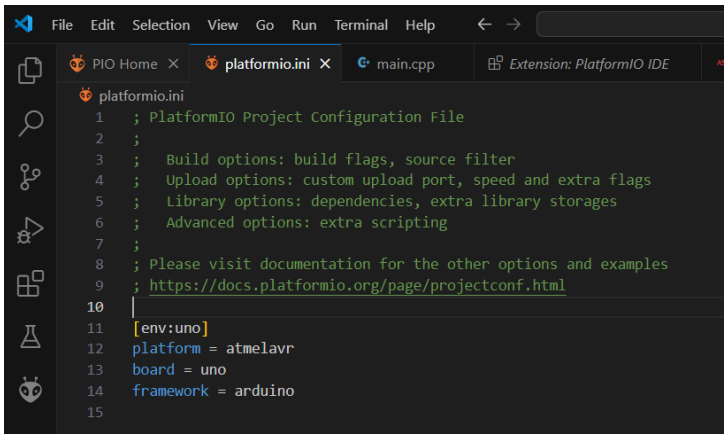
IV. Results and Observations

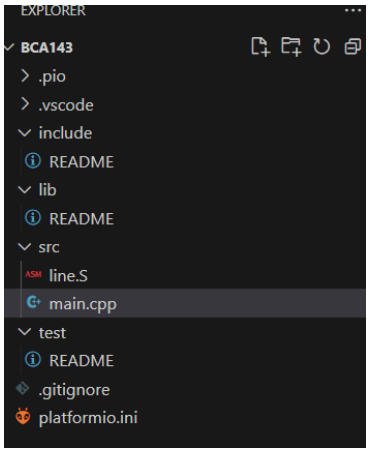
During the firmware development process, the researchers discovered that the Arduino IDE does not natively support or automatically detect Assembly code. To address this, the Assembly code had to be embedded within an Embedded C file before it could be executed properly (as shown in the screenshot below). Additionally, toolchain compatibility issues were encountered with the latest version of the Arduino IDE, making debugging and troubleshooting more challenging due to inconsistencies and limitations in the supported toolchain components.











Successful code compilation upon build time and flashing the program to the ATMega32P main memory using proper platform configuration.

When the motor speed is set too high in the code, it significantly reduces the time available for the sensors to accurately detect the line. As a result, the robot may misinterpret or entirely miss critical tracking data, leading to unstable or erratic movement. This can cause the robot to veer off course, lose alignment with the track, and ultimately fail to follow the intended path. Proper tuning of the motor speed is therefore essential to ensure that the sensors have enough time to process data accurately and maintain smooth and consistent line-following behavior.

V. Conclusions and Recommendations

Summary of the project's success

A functional line-following robot was designed and constructed, integrating an Arduino Uno, L298N H-Bridge motor driver, two IR proximity sensors, and DC motors, all powered by a LiPo battery. The implementation of the line-following control algorithm in 8-bit AVR Assembly language, was accomplished. This involved translating initial C code logic to assembly (line.S) and utilizing PlatformIO for compilation and deployment,

demonstrating proficiency in low-level hardware programming.Lastly it demonstrated\ interfacing between the ATMega328P microcontroller and various peripheral components. The robot successfully utilized IR sensors for line detection and controlled motor speeds and direction via the L298N.

Possible enhancements for future versions

- Incorporate ultrasonic sensors or additional IR range sensors (as suggested by related literature on obstacle detection) to enable the robot to detect and potentially navigate around obstacles, rather than just stopping or recovering from line gaps.
- Increase the number of IR sensors in an array for more precise line tracking, especially on complex paths with sharp curves, intersections, or varying line widths. Encoders for - Add rotary encoders to the motors for closed-loop speed control, allowing for more accurate and consistent movement.
- Transition from a breadboard/protoboard setup to a custom-designed Printed Circuit Board (PCB)

- Manually adjusting the motor speed, delay values, and other parameters in the code is essential and must be tailored to the physical structure of the robot. Factors such as the robot's overall weight, weight distribution, and balance significantly affect its movement and performance. For instance, robots that are too heavy, too light, or unevenly weighted may experience issues like instability, flipping, or lifting at the front

VI. References

Bajestani, S. E. M., & Vosoughinia, A. (2010). Technical report of building a line follower robot. *2010 International Conference on Electronics and Information Engineering*, *1*, V1-1-V1-5. https://doi.org/10.1109/ICEIE.2010.5559826

Barua, V., Arif, Md., Uddin, S., Das, M., Shafiul, Md., Nahar, N., & Pathak, A. (2020). An Ultrasonic Line Follower Robot to Detect Obstacles and Edges for Industrial and Rescue Operations. *International Journal of Computer Applications*, *176*(31), 31–37. https://doi.org/10.5120/ijca2020920364

Colak, I., & Yildirim, D. (2009). Evolving a Line Following Robot to use in shopping centers for entertainment. *2009 35th Annual Conference of IEEE Industrial Electronics*, 3803–3807. https://doi.org/10.1109/IECON.2009.5415369

Gadekar, S., Kolpe, G., Rutuja, G., Fatate, V., Bhosale, B., Chate, S., & Lad, A. (2021). *Arduino Uno-ATmega328 P Microcontroller Based Smart Systems* (SSRN Scholarly Paper No. 3920231). Social Science Research Network. https://doi.org/10.2139/ssrn.3920231

Ghani, N. M., Naim, F., & Yon, T. P. (2011). *Two wheels balancing robot with line following capability*. *79*, 634–638.

Gomi, T., & Ide, K. (1998). Evolution of gaits of a legged robot. *1998 IEEE International Conference on Fuzzy Systems Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98CH36228)*, *1*, 159–164 vol.1. https://doi.org/10.1109/FUZZY.1998.687476

He, X., Wang, Z., Fang, H., He, K., & Du, R. (2014). An embedded robot controller based on ARM and FPGA. *2014 4th IEEE International Conference on Information Science and Technology*, 702–705. https://doi.org/10.1109/ICIST.2014.6920574

Jain, T., Sharma, R., & Chauhan, S. (2014). *Applications of Line Follower Robot in Medical Field*. *1*(11).

Kaiser, F., Islam, S., Imran, W., Khan, K. H., & Islam, K. M. A. (2014). Line follower robot: Fabrication and accuracy measurement by data acquisition. *2014 International Conference on Electrical Engineering and Information & Communication Technology*, 1–6. https://doi.org/10.1109/ICEEICT.2014.6919137

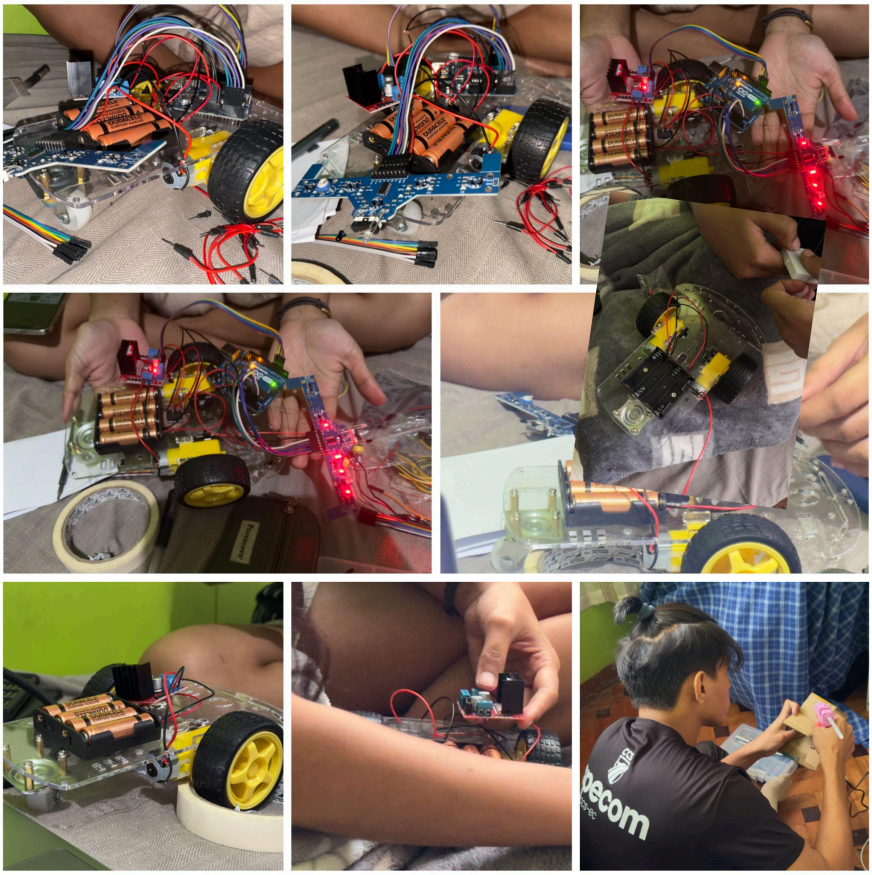
Kokare, S., Shastri, R., & Kolhar, S. (2018). Line Follower with Obstacle Information System Using ZigBee. *2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA)*, 1–5. https://doi.org/10.1109/ICCUBEA.2018.8697470

Latif, A., Widodo, H. A., Rahim, R., & Kunal, K. (2020). Implementation of Line Follower Robot based Microcontroller ATMega32A. *Journal of Robotics and Control (JRC)*, *1*(3), Article 3. https://doi.org/10.18196/jrc.1316

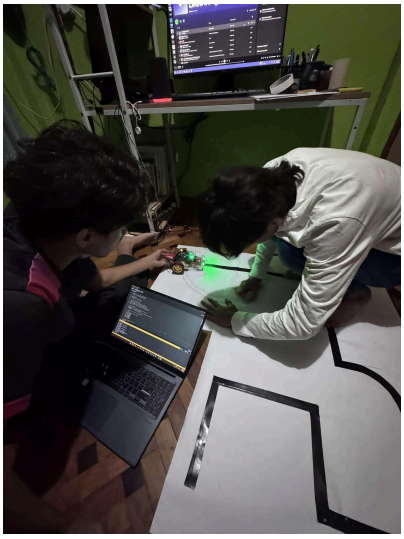
Pakdaman, M., & Sanaatiyan, M. M. (2009). Design and Implementation of Line Follower Robot. *2009 Second International Conference on Computer and Electrical Engineering*, *2*, 585–590. https://doi.org/10.1109/ICCEE.2009.43

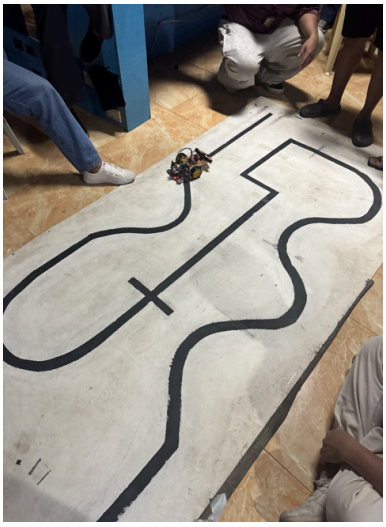
Rafi, R. H., Das, S., Ahmed, N., Hossain, I., & Taslim Reza, S. M. (2016). Design & implementation of a line following robot for irrigation based application. *2016 19th International Conference on Computer and Information Technology (ICCIT)*, 480–483. https://doi.org/10.1109/ICCITECHN.2016.7860245

APPENDIX

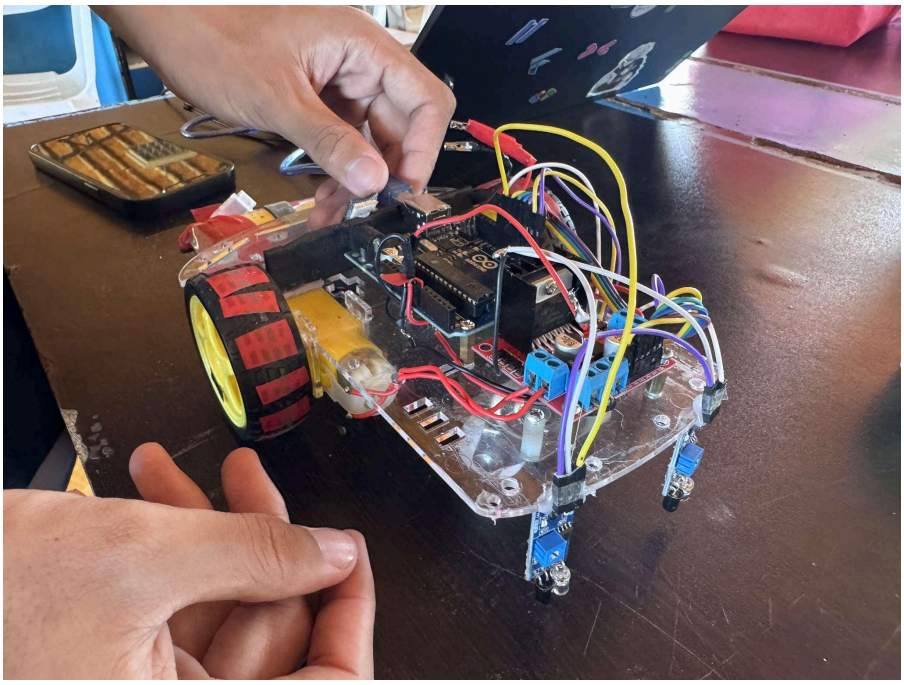


Hardware Development of the Line Following Robot

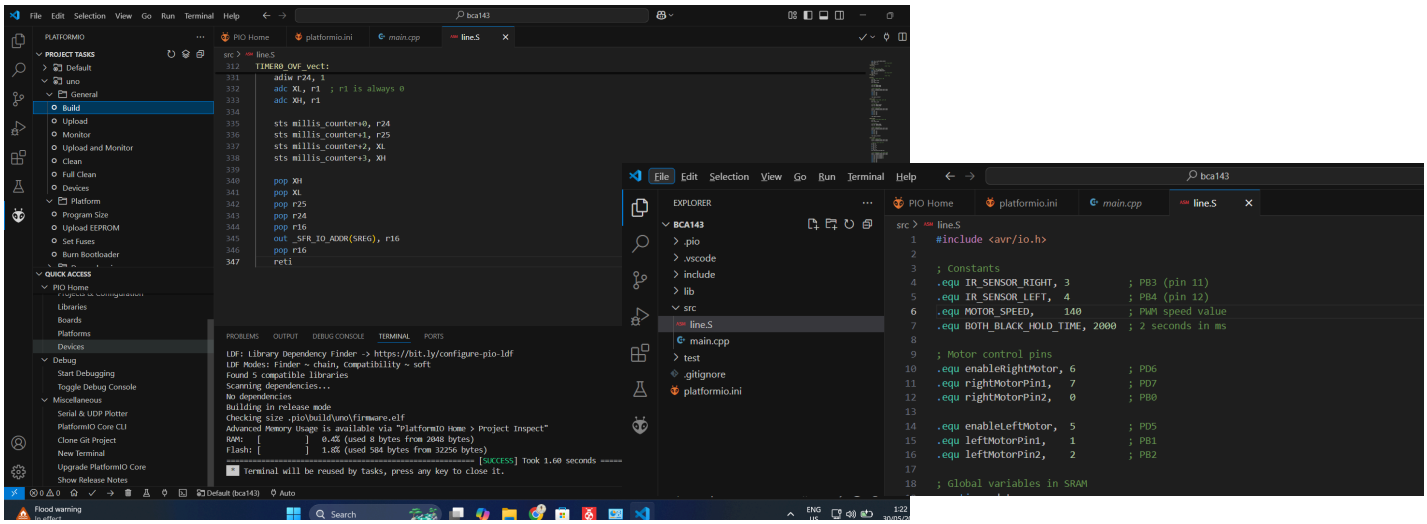




Initial Line Tracking Testing



Developed Line Following Robot



Firmware Development using PlatformIO

```
extern "C" {
    void asm_main();
    void TIMER0_OVF_vect(void)
__attribute__ ((signal));
}

// Weak dummy functions to satisfy
linker
void setup() __attribute__ ((weak));
void loop() __attribute__ ((weak));
void setup() {}
void loop() {}
```

```
// Custom entry point
__attribute__((naked, used,
section(".init3")))
void _init3() {
    asm volatile(
        "clr __zero_reg__\n\t"
        "jmp asm_main"
    );
}
```

Main.cpp code

```
#include <avr/io.h>

; Constants
.equ IR_SENSOR_RIGHT, 3          ;
PB3 (pin 11)
.equ IR_SENSOR_LEFT,  4          ;
PB4 (pin 12)
.equ MOTOR_SPEED,     140        ;
PWM speed value
.equ BOTH_BLACK_HOLD_TIME, 2000  ; 2
seconds in ms

; Motor control pins
.equ enableRightMotor, 6         ;
PD6
.equ rightMotorPin1,   7         ;
PD7
.equ rightMotorPin2,   0         ;
PB0

.equ enableLeftMotor,  5         ;
PD5
.equ leftMotorPin1,    1         ;
PB1
.equ leftMotorPin2,    2         ;
PB2

; Global variables in SRAM
.section .data
millis_counter:          .space 4
both_black_start_time:   .space 4

.section .vectors,"a",@progbits
    .weak __vector_16
    .set __vector_16, TIMER0_OVF_vect

; Program code
.section .text
.global asm_main

asm_main:
    ; Initialize stack pointer
    ldi r16, hi8(RAMEND)
    out _SFR_IO_ADDR(SPH), r16
    ldi r16, lo8(RAMEND)
    out _SFR_IO_ADDR(SPL), r16

    ; Initialize SRAM variables to 0
    ldi YL, lo8(millis_counter)
    ldi YH, hi8(millis_counter)
    clr r16
    std Y+0, r16
    std Y+1, r16
    std Y+2, r16
    std Y+3, r16

    ldi YL,
lo8(both_black_start_time)
    ldi YH,
hi8(both_black_start_time)
    std Y+0, r16
    std Y+1, r16
    std Y+2, r16
    std Y+3, r16

    ; Set Timer0 for 1ms interrupts
(prescaler 64)
    ldi r16, (1<<CS00) | (1<<CS01)   ;
Prescaler 64
    out _SFR_IO_ADDR(TCCR0B), r16
    ldi r16, (1<<TOIE0)              ;
Enable overflow interrupt
    sts TIMSK0, r16
    ldi r16, 6                       ;
Preload for 250 counts (256-250=6)
    out _SFR_IO_ADDR(TCNT0), r16

    ; Set Timer0 prescaler for PWM
    in r16, _SFR_IO_ADDR(TCCR0B)
    andi r16, 0xF8
    ori r16, (1<<CS01)               ;
Set prescaler to 8
    out _SFR_IO_ADDR(TCCR0B), r16

    ; Configure motor control pins as
outputs
```

```
    sbi _SFR_IO_ADDR(DDRD),
enableRightMotor
    sbi _SFR_IO_ADDR(DDRD),
rightMotorPin1
    sbi _SFR_IO_ADDR(DDRB),
rightMotorPin2

    sbi _SFR_IO_ADDR(DDRD),
enableLeftMotor
    sbi _SFR_IO_ADDR(DDRB),
leftMotorPin1
    sbi _SFR_IO_ADDR(DDRB),
leftMotorPin2

    ; Configure sensor pins as inputs
    cbi _SFR_IO_ADDR(DDRB),
IR_SENSOR_RIGHT
    cbi _SFR_IO_ADDR(DDRB),
IR_SENSOR_LEFT

    ; Initialize PWM
    ldi r16, (1<<COM0A1) |
(1<<COM0B1) | (1<<WGM00) | (1<<WGM01)
    out _SFR_IO_ADDR(TCCR0A), r16

    ; Initialize motors to stop
    clr r20
    clr r21
    rcall rotate_motor

    ; Enable global interrupts
    sei

main_loop:
    ; Read right sensor (PB3)
    in r16, _SFR_IO_ADDR(PINB)
    andi r16, (1<<IR_SENSOR_RIGHT)
    breq right_low
    ser r18                    ; Set
flag (0xFF)
    rjmp check_left
right_low:
    clr r18                    ; Clear
flag
```

```
check_left:
    ; Read left sensor (PB4)
    in r16, _SFR_IO_ADDR(PINB)
    andi r16, (1<<IR_SENSOR_LEFT)
    breq left_low
    ser r19                    ; Set
flag (0xFF)
    rjmp evaluate
left_low:
    clr r19                    ; Clear
flag

evaluate:
    ; Case 1: Both sensors LOW
    tst r18
    brne check_case2
    tst r19
    brne check_case3

    ; Go straight
    ldi r20, MOTOR_SPEED
    ldi r21, MOTOR_SPEED

    ; Reset start time
    ldi YL,
lo8(both_black_start_time)
    ldi YH,
hi8(both_black_start_time)
    clr r16
    std Y+0, r16
    std Y+1, r16
    std Y+2, r16
    std Y+3, r16

    rjmp apply_movement

check_case2:
    ; Case 2: Right HIGH, Left LOW
    tst r19
    brne check_case4

    ; Turn right
    ldi r20, -MOTOR_SPEED
```

```
    ldi r21, MOTOR_SPEED

    ; Reset start time
    ldi YL,
lo8(both_black_start_time)
    ldi YH,
hi8(both_black_start_time)
    clr r16
    std Y+0, r16
    std Y+1, r16
    std Y+2, r16
    std Y+3, r16

    rjmp apply_movement

check_case3:
    ; Case 3: Left HIGH, Right LOW
    tst r18
    brne check_case4

    ; Turn left
    ldi r20, MOTOR_SPEED
    ldi r21, -MOTOR_SPEED

    ; Reset start time
    ldi YL,
lo8(both_black_start_time)
    ldi YH,
hi8(both_black_start_time)
    clr r16
    std Y+0, r16
    std Y+1, r16
    std Y+2, r16
    std Y+3, r16

    rjmp apply_movement

check_case4:
    ; Case 4: Both HIGH
    ldi YL,
lo8(both_black_start_time)
    ldi YH,
hi8(both_black_start_time)
```

```
    ; Check if start time is 0
    ld r16, Y+
    ld r17, Y+
    or r16, r17
    ld r17, Y+
    or r16, r17
    ld r17, Y
    or r16, r17
    brne both_black_already_set

    ; First detection - set start
time
    ldi YL,
lo8(both_black_start_time)
    ldi YH,
hi8(both_black_start_time)
    lds r22, millis_counter+0
    lds r23, millis_counter+1
    lds r24, millis_counter+2
    lds r25, millis_counter+3
    std Y+0, r22
    std Y+1, r23
    std Y+2, r24
    std Y+3, r25

    ; Stop motors
    clr r20
    clr r21
    rjmp apply_movement

both_black_already_set:
    ; Calculate time difference
    lds r22, millis_counter+0
    lds r23, millis_counter+1
    lds r24, millis_counter+2
    lds r25, millis_counter+3

    ldi YL,
lo8(both_black_start_time)
    ldi YH,
hi8(both_black_start_time)
    ld r18, Y+
    ld r19, Y+
    ld r20, Y+
```

```
    ld r21, Y

    ; Compute difference (current -
start)
    sub r22, r18
    sbc r23, r19
    sbc r24, r20
    sbc r25, r21

    ; Check if difference >= 2000 ms
    ldi r18,
lo8(BOTH_BLACK_HOLD_TIME)
    ldi r19,
hi8(BOTH_BLACK_HOLD_TIME)
    ldi r20, 0
    ldi r21, 0

    ; Compare difference with 2000
    cp r22, r18
    cpc r23, r19
    cpc r24, r20
    cpc r25, r21
    brsh time_elapsed

    ; Time not elapsed - stop motors
    clr r20
    clr r21
    rjmp apply_movement

time_elapsed:
    ; Move forward after 2 seconds
    ldi r20, MOTOR_SPEED
    ldi r21, MOTOR_SPEED

apply_movement:
    rcall rotate_motor
    rjmp main_loop

rotate_motor:
    ; Right motor direction control
    tst r20
    breq right_stop
    brmi right_backward

    ; Forward
    sbi _SFR_IO_ADDR(PORTD),
rightMotorPin1
    cbi _SFR_IO_ADDR(PORTD),
rightMotorPin2
    rjmp set_right_speed

right_backward:
    ; Backward
    cbi _SFR_IO_ADDR(PORTD),
rightMotorPin1
    sbi _SFR_IO_ADDR(PORTD),
rightMotorPin2
    rjmp set_right_speed

right_stop:
    cbi _SFR_IO_ADDR(PORTD),
rightMotorPin1
    cbi _SFR_IO_ADDR(PORTD),
rightMotorPin2

set_right_speed:
    ; Set PWM value (absolute)
    mov r16, r20
    tst r16
    brpl right_abs
    neg r16
right_abs:
    out _SFR_IO_ADDR(OCR0A), r16

    ; Left motor direction control
    tst r21
    breq left_stop
    brmi left_backward

    ; Forward
    sbi _SFR_IO_ADDR(PORTB),
leftMotorPin1
    cbi _SFR_IO_ADDR(PORTB),
leftMotorPin2
    rjmp set_left_speed

left_backward:
    ; Backward
```

```
    cbi _SFR_IO_ADDR(PORTB),
leftMotorPin1
    sbi _SFR_IO_ADDR(PORTB),
leftMotorPin2
    rjmp set_left_speed

left_stop:
    cbi _SFR_IO_ADDR(PORTB),
leftMotorPin1
    cbi _SFR_IO_ADDR(PORTB),
leftMotorPin2

set_left_speed:
    ; Set PWM value (absolute)
    mov r16, r21
    tst r16
    brpl left_abs
    neg r16
left_abs:
    out _SFR_IO_ADDR(OCR0B), r16
    ret

; Timer0 overflow interrupt (1ms)
.global TIMER0_OVF_vect
TIMER0_OVF_vect:
    push r16
    in r16, _SFR_IO_ADDR(SREG)
    push r16
    push r24
    push r25
    push XL
    push XH

    ; Preload for 250 counts
(256-250=6)
    ldi r16, 6
    out _SFR_IO_ADDR(TCNT0), r16

    ; Increment millis counter
(32-bit)
    lds r24, millis_counter+0
    lds r25, millis_counter+1
    lds XL, millis_counter+2
    lds XH, millis_counter+3

    adiw r24, 1
    adc XL, r1  ; r1 is always 0
    adc XH, r1

    sts millis_counter+0, r24
    sts millis_counter+1, r25
    sts millis_counter+2, XL
    sts millis_counter+3, XH

    pop XH
    pop XL
    pop r25
    pop r24
    pop r16
    out _SFR_IO_ADDR(SREG), r16
    pop r16
    reti
```

Line.S Code (8 bit AVR Assembly)

VIDEO DEMO
https://drive.google.com/drive/folders/1ktsXRBvpUQ1DoTdpzVVPgW9WNUi8Hlqo